

Chapter 8

Indexing

by Lorraine Gaudio

Version June 03, 2026

Contents

1	Overview	2
2	Activity Setup	2
2.1	Create/Save Chapter 8 Notebook	2
2.2	Downloading Data	3
2.3	Load Data	3
3	Subsetting	4
3.1	Positional Indexing	5
3.2	Names Indexing	7
3.3	Logical Mask Indexing	8
4	Extraction	12
4.1	[[Operator	12
4.2	\$ Operator	13
5	Subassignment (replacement)	14
5.1	Overwrite with <code>x[i, j] <- value</code>	15
5.2	Insertion	16
6	Code Chunk Options	18
7	Summary	19
8	Chapter Terms	19
9	Practice Space	20
9.1	Task 1	21
9.2	Task 2	22
9.3	Task 3	22
9.4	Task 4	22
9.5	Task 5	23
9.6	Task 6	23
9.7	Task 7	23
9.8	Task 8	24
9.9	Task 9	25

1 Overview

Bracket indexing is one of the main ways you tell R exactly which rows, columns, or values you want to use. Real data analysis rarely uses an entire dataset all at once. Analysts often need to select a few columns, filter rows, extract one column for a summary, correct data-entry problems, or create a new column for later analysis.

In this chapter, you will practice those skills using the `field_sightings` dataset, a simulated bird-banding dataset where birds are recorded as Tagged or Untagged, and tagged birds can have a band color. The data frame has two columns: `Status` and `Band_Color`. In the Practice Space, you will apply the same indexing patterns to `Values_Transport_dplace.csv` using columns such as `id`, `name`, `sub_case`, and `year`.

These skills matter because indexing is how you point R at the part of the data you actually mean. If you select the wrong rows, extract the wrong kind of object, or overwrite values without checking first, later analysis can become unreliable. This chapter helps you build the habit of checking what your code returned before moving on.

By the end of Chapter 8, you will be able to:

- Use a loaded CSV file as a data frame in R. Apply the subsetting template `df[rows, cols]` to select data by position, name, and logical mask.
- Check whether your indexing code returned the rows, columns, object type, and values you expected.
- Explain how missing values can propagate through logical masks and create confusing filtering results.
- Use `which()` to turn logical conditions into row positions when you want to avoid carrying NA values into a subset.
- Extract one column as “the thing itself” using `[[ ]]` or `$` so you can pass the values into functions like `unique()` or `table()`.
- Use subassignment to overwrite selected values in a copied data frame and insert a new column.
- Use code chunk options to manage long outputs and make your R Notebook easier to read.

2 Activity Setup

Chapter 7 introduced you folder organization and how to import data into R. This section will lightly review what you learned, providing you with additional practice. If you need a more detailed refresher on how to import data, start by reviewing Chapter 7 Importing Data.

2.1 Create/Save Chapter 8 Notebook

Create an R Notebook, edit the YAML, and save in `Module_4/`

1. Open an R Notebook: Go to **File > New File > R Notebook**
2. Edit the YAML Header and delete the default text. The YAML header should look like this:

```
---
title: "Chapter 8 Notes"
author: "[Your Name]"
date: "`r Sys.Date()`"
output: html_notebook
---
```

3. Change `[Your Name]` to your actual name and clear the default text in the body of the notebook.
4. Save your R Notebook **File > Save As...**, choose your course folder, name the file `chapter8_notes`.

Suggested file structure:

```
Intro_to_R/
  Module_4/
```

```
chapter8_notes.Rmd # Add the guided activity file to your `Module_4/` folder
chapter7_practice.Rmd
Values_Transport_dplace.csv
car_prices.xlsx
```

2.2 Downloading Data

You will need two data files for this chapter:

- Values_Transport_dplace.csv
- field_sightings.csv

You should already have Values_Transport_dplace.csv in your folder. Find the field_sightings.csv data file in the **Indexing Guided Activity** instructions page.

Click the download link. **Do not open the downloaded csv.**

Move the CSV into your Module_4/ folder with your Rmd file.

```
Intro_to_R/
Module_4/
  chapter8_notes.Rmd
  chapter7_practice.Rmd
  Values_Transport_dplace.csv
  car_prices.xlsx
  field_sightings.csv # Add the CSV file to your `Module_4/` folder
```

If your folder has a different structure, that is okay as long as you can explain where your Rmd file is saved and how to address the file path to the data files. The important habit is knowing where your work lives before you try to load data. For novices, it is easiest to place the data files in the same folder as the Rmd file.

Now that you have your R Notebook set up and the data files in place, you are ready to load the data.

2.3 Load Data

The field_sightings.csv dataset contains **simulated** (randomly generated) values. The dataset theme is based on the pretend situation where field biologists observed and recorded a population of species of birds. Bird banding is a scientific method used to track and study individual birds, providing valuable data on their movements, behavior, and population dynamics. In our scenario, most birds observed were “Untagged,” but a small number have been previously captured and marked with colored leg bands, “Tagged”. The color of the bands might provide information about the bird’s history, such as where it was banded or its age.

The simulated dataframe field_sightings has two columns:

- **Status** Describes whether the sighted bird is “Untagged” (the common case) or “Tagged” (the rare case).
- **Band_Color** If a bird is untagged, the value is “None.” If it is tagged, the value is one of several possible band colors.

```
# Type this in your R Notebook
# 1) Confirm the CSV is in the same folder as your R Notebook
list.files()
```

Look in the output from list.files(). Confirm that field_sightings.csv appears exactly as written, including the underscore and .csv ending.

If `field_sightings.csv` is not listed, check whether the file is still in `Downloads/`, whether it was renamed, or whether it was saved in a different folder. Your next move is to make the file visible to R in your Rmd, then run `list.files()` again.

Load `field_sightings.csv` into R using `read_csv()` and assign it to a variable called `field_sightings`. Recall that you need to `library(readr)` before you can use `read_csv()`.

```
# Type this in your R Notebook
library(readr) # Load the readr package for reading CSV files
```

```
## Warning: package 'readr' was built under R version 4.5.3
```

```
field_sightings <- read_csv("field_sightings.csv")
```

```
## Rows: 4470 Columns: 2
## -- Column specification -----
## Delimiter: ","
## chr (2): Status, Band_Color
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

This code uses the `read_csv()` function to read the CSV file named “`field_sightings.csv`” and assigns the resulting data frame to a variable called `field_sightings`. It is normal to see messages with you load packages.

Look in the Environment pane for `field_sightings`. Then run `head(field_sightings)` to confirm that the data has two columns, `Status` and `Band_Color`.

```
# Check
head(field_sightings)
```

```
## # A tibble: 6 x 2
##   Status   Band_Color
##   <chr>    <chr>
## 1 Untagged None
## 2 Untagged None
## 3 Untagged None
## 4 Untagged None
## 5 Untagged None
## 6 Untagged None
```

Feel free to click `field_sightings` in your Environment pane to view the data frame and get a sense of what the data looks like.

Test knit. Before you move on, test knit your R Notebook to make sure everything is working. Check that the formatting is working as expected (e.g., headings appear as heading font, code appears in code chunks, and memos are paragraph text)

With your data loaded in to the environment, you are ready to start practicing subsetting, extraction, and replacement techniques in R.

3 Subsetting

Subsetting a smaller part of an object, typically using *positions* (integers), *names* (characters), or *logical mask* (TRUE/FALSE). In base R, this is done using the **single square bracket operator** `[ ( [ ) ]`.

The return output from a single square bracket operator is usually a container of the same type as the original object. A subset of a vector is still a vector and the subset of a data frame is often still a data frame. Data analysts use subsetting when filtering rows, selecting columns to keep, slicing a range, or pulling a rectangular block from a data frame.

3.1 Positional Indexing

Positional indexing is the most basic form of indexing in R, where you use numeric positions to select elements from a vector or data frame.

Tip: Positions start at 1 in R. This is different from other coding languages like python which starts indexing values at 0.

3.1.1 Vector

In Chapter 3, you practiced using the **indexing operator** with positional indexing to extract elements from vectors. Let's review that briefly before we move on to data frames.

You learned that the basic pattern for **positional indexing** is `x[i]`, where `x` is the vector and `i` is the index (or indices) of the elements you want to extract. A position used to identify elements in an object.

Practice using `[ ]` with positional indexing on vectors.

```
# Review
z <- c(10, 20, 30, 40, 50) # create a vector
```

```
# Check
print(z)
```

```
## [1] 10 20 30 40 50
```

```
# Review
z[1]
```

```
## [1] 10
```

The first element by position is 10.

```
# Review
z[3]
```

```
## [1] 30
```

The third element by position is 30.

```
# Review
z[2:4]
```

```
## [1] 20 30 40
```

The second, third and fourth elements by position is 20, 30, and 40.

3.1.2 Data Frame

The **indexing operator** (`[ ]`) is used to select or replace parts of many kinds of objects, not just vectors. For data frames, the common form is `df[rows, cols]` or `x[i, j]` where `i` is the **row index**, `j` is the **column index**. If you leave one side blank, the output includes all.

Rows and columns are coordinates:

- `df[1:5, ]` results in the first five rows, all columns
- `df[, 1:3]` shows all rows, first three columns

- `df[1, 2]` prints only one cell's value: first row, second column

Let's see this in practice with our `field_sightings` data frame.

Print the first six rows of the `field_sightings` data frame using the indexing operator.

Before running a bracket-indexing line below, pause and ask: "How do I know that this is choosing rows?"

```
# Type this in your R Notebook
field_sightings[1:6, ]
```

```
## # A tibble: 6 x 2
##   Status   Band_Color
##   <chr>    <chr>
## 1 Untagged None
## 2 Untagged None
## 3 Untagged None
## 4 Untagged None
## 5 Untagged None
## 6 Untagged None
```

Tip: I remember bracket indexing by watching the comma. In `df[rows, columns]`, the comma divides the row instruction from the column instruction. The left side answers "which rows?" The right side answers "which columns?" **In `df[rows, cols]`, the comma is the divider.**

Check that the output has 6 rows and both columns. The blank space after the comma means "keep all columns."

This shows the first six rows of `field_sightings` while keeping all columns.

3.1.3 `head()` Function *Comparison*

There is a function called `head()` that *basically* does the same thing as `field_sightings[1:6, ]` but is more concise and easier to read.

The SYNTAX:

```
head(x, n = 6L)
```

If you don't specify `n`, it defaults to 6, which means it will show the first 6 rows of the data frame. You can change this number to show more or fewer rows.

Print the first three rows of the `field_sightings` data frame using the `head()` function to check that the dataset loaded correctly.

```
# Type this in your R Notebook
head(field_sightings, n = 3)
```

```
## # A tibble: 3 x 2
##   Status   Band_Color
##   <chr>    <chr>
## 1 Untagged None
## 2 Untagged None
## 3 Untagged None
```

Confirm that the output shows exactly 3 rows and the same column names as the full data frame.

`head()` is commonly used to get a quick orientation of the dataset after loading.

`head()` does not do arbitrary extraction or subsetting. `head()` is a function that provides a convenient way to view the first few rows of a data frame.

Use `head()` when you want a quick look at the beginning of a dataset. Use bracket indexing when you need to select specific rows, columns, or conditions.

Test knit.

Bracket indexing allows for flexible and specific subsetting of the data frame based on row *and* column indices. `[]` works on almost everything and supports rich indexing and is commonly used as part of an algorithm. You will use `[]` when you need subsetting by columns, conditions, and arbitrary rows when you are writing analysis code.

3.2 Names Indexing

You can also use **names indexing** (also referred to as **character indexing**) to select columns by their names instead of their positions. This often makes more readable code and is less error-prone, especially when the order of columns might change or when you have many columns.

3.2.1 Vector

In Chapter 3, you created the `nerdy_palette` vector and then used **names indexing** to return the hexadecimal color code.

```
# Review
# Create nerdy_palette
nerdy_palette <- c(
  "Octarine"      = "#7F00FF",
  "TARDIS Blue"   = "#003B6F",
  "Rebeccapurple" = "#663399",
  "Hoolooovoo"    = "#4F97A3",
  "Cosmic Latte"  = "#FF8E7",
  "Vantablack"    = "#000000",
  "Matrix Green"  = "#03A062",
  "Source Engine Magenta" = "#FF00FF",
  "Command Red"   = "#CC0000",
  "Impossible Green" = "#4D9E68"
)

# Use names indexing to return the hex code for Octarine
nerdy_palette["Octarine"]

## Octarine
## "#7F00FF"
```

The output shows the hexadecimal color code for “Octarine” retrieved from the `nerdy_palette` vector using its name.

3.2.2 Data Frame

A common use of names indexing is to **select a column by its name**.

Use names indexing to return the band colors in the `field_sightings` data frame.

Methods A:C show different ways to subset a column with `[]`.

Method A

The SYNTAX: `x[i]`

```
# Type this in your R Notebook
field_sightings["Band_Color"]      # 1-column DATA FRAME
```

Check the output structure.

A 1-column data frame is printed showing all row values in the column `Band_Color` the `field_sightings` data frame. It is a subset of one column, keeping that data-frame structure.

Method B

The SYNTAX: `df[rows, cols]`

```
# Type this in your R Notebook
# Example B
field_sightings[, "Band_Color"]    # usually the column VECTOR
```

Check the output structure.

A vector is printed showing all values from the column `Band_Color` the `field_sightings` data frame. The result is a subset from one column and returns a vector, values only.

Method C

The SYNTAX: `x[i, j, drop = FALSE]`

```
# Type this in your R Notebook
# Example C
field_sightings[, "Band_Color", drop = FALSE]    # 1-column DATA FRAME
```

Check the output structure.

A 1-column data frame is printed showing all row values in the column `Band_Color` the `field_sightings` data frame. It is a subset of one column, keeping that data-frame structure.

Common Pain Point: Students often forget to put the column name in quotes when using names indexing. If you forget the quotes, R will look for an object named `Band_Color` in your environment and return an error if it doesn't find one. Always remember to put column names in quotes when using **names indexing** with `[ ]`.

The autonomous move here is not memorizing three methods as separate tricks. It is deciding what kind of output you need next. If you need a smaller data frame, keep the data-frame structure. If you need values to pass into a function, extract the column as a vector.

Test knit.

So far, you've learned about positional indexing and names indexing. Both of these methods rely on you knowing the positions or names of the elements you want to subset. But what if you want to subset based on a condition, like keeping only rows where `Status` is "Tagged"? This is where **logical mask indexing** comes in.

3.3 Logical Mask Indexing

A **logical mask** is the logical vector you use to index.

3.3.1 Vector

In Chapter 4, you practiced logical indexing with a vector.

Create a new vector called `smaller` that contains only values in `number_list` that are less than or equal to 20.

```
# Type this in your R Notebook
# Create the sample vector
number_list <- c(10, 25, 5, 30, 15, 40)
```

```
# Check
print(number_list)
```

```
## [1] 10 25  5 30 15 40
```

The `number_list` vector contains the values 10, 25, 5, 30, 15, and 40.

```
# Type this in your R Notebook
# Example 1
# Step 1: create the logical "mask"
mask <- number_list <= 20
```

```
# Check
print(mask)
```

```
## [1] TRUE FALSE TRUE FALSE TRUE FALSE
```

Check that `mask` has the same number of values as `number_list`. Each value should be `TRUE` or `FALSE`, depending on whether the matching number is less than or equal to 20.

This prints the `mask`. The mask itself is not a subset of the original vector. The logical vector indicates which values in `number_list` are less than or equal to 20.

```
# Type this in your R Notebook
# Example 1
# Step 2: use the logical vector inside [ ]
smaller <- number_list[mask]
```

```
# Check
print(smaller)
```

```
## [1] 10  5 15
```

Check that `smaller` contains only the values from `number_list` where the mask was `TRUE`: 10, 5, and 15.

This keeps only the values where the condition is `TRUE` (10, 5, 15).

You can also pass the mask directly into the indexing operator without storing it in a variable first.

```
# Type this in your R Notebook
# Example 2
smaller <- number_list[number_list <= 20]
```

Both example 1 and example 2 are **conditional subsetting** or filtering by using a logical condition. These examples use the **less than or equal to (`<=`) relational operator**. The **logical mask** is the result of the comparison (`number_list <= 20`).

`[ ]` uses it as a filter. **Logical mask indexing** is a form of indexing in R where you use a logical vector (a “mask” of `TRUE`/`FALSE` values) to select elements from a vector or rows from a data frame.

R keeps the values where the mask is `TRUE` and drops the values where the mask is `FALSE`.

Test knit.

Review: Chapter 3 introduced comparison operators.

Operator	Meaning	Example
>	greater than	mpg > 20
<	less than	mpg < 15
==	equal to	am == 1
>=	greater <i>or</i> equal	mpg >= 25
<=	less <i>or</i> equal	mpg <= 15
!	not	!(gear %in% c(3,4))
!=	not equal to	gear != 4

Storing the mask first (example 1) is useful when you want to inspect or reuse the logic. Putting the condition directly inside `[ ]` (example 2) is useful when the condition is short and easy to read. Both are valid.

Let's see this in action in a dataframe.

3.3.2 Data Frame

Logical mask indexing works the same way for data frames, except that the logical mask *usually* goes in the **rows** position of `df[rows, cols]`.

The mask can be TRUE/FALSE/NA for each row. When you use the mask in `df[mask, ]`, R keeps rows where the mask is TRUE. If the mask contains NA, those unknown positions can also appear in the result, which can make the subset larger than the number of definite TRUE values.

: Keep only rows where the bird is tagged using `[ ]`.

```
# Type this in your R Notebook
# Step 1: create the logical mask (one TRUE/FALSE per row)
mask_tagged <- field_sightings[, "Status"] == "Tagged"
```

The `==` is a comparison operator that checks if the value in the **Status** column is equal to "Tagged". A single `=` is used for assigning values inside some function calls (like `na.rm = TRUE`) and naming arguments. It is not used to test equality.

Check the first few values with `head(mask_tagged)`. You should see a logical vector containing TRUE, FALSE, and possibly NA.

```
# Check
head(mask_tagged)
```

```
##      Status
## [1,] FALSE
## [2,] FALSE
## [3,] FALSE
## [4,] FALSE
## [5,] FALSE
## [6,] FALSE
```

This prints a logical mask the same length as the number of rows in `field_sightings`. Each TRUE means this row has `Status == 'Tagged'`.

```
# Type this in your R Notebook
# Step 2: use the logical mask inside [ ] to subset rows
tagged_only <- field_sightings[mask_tagged, ]

# Alternative: pass the logical mask directly into [ ]
# tagged_only <- field_sightings[field_sightings[, "Status"] == "Tagged", ]
```

Look at the Status column in the first few rows of `tagged_only`. You should mostly see “Tagged”, but if the mask contained NA, you may also see rows with missing values.

```
# Check
head(tagged_only)

## # A tibble: 6 x 2
##   Status Band_Color
##   <chr>   <chr>
## 1 Tagged Orange
## 2 Tagged Silver
## 3 Tagged Green
## 4 Tagged Black
## 5 Tagged Silver
## 6 Tagged Blue
```

This subsets the data frame by keeping only rows where the mask is TRUE.

Compare the three counts. `nrow(field_sightings)` tells you the size of the original data frame. `nrow(tagged_only)` tells you how many rows were returned by the subset. `sum(mask_tagged, na.rm = TRUE)` tells you how many rows were definitely TRUE.

```
# Check
nrow(field_sightings)      # how many rows in the original data frame?

## [1] 4470

nrow(tagged_only)          # how many tagged rows?

## [1] 325

sum(mask_tagged, na.rm= TRUE) # how many TRUE values in the mask?

## [1] 300
```

The result is a smaller data frame containing only the tagged birds. There are NA values, so we needed to use `na.rm = TRUE` to get an accurate count only the TRUE values. We can tell the mask contains NA values because `nrow(tagged_only)` (325) is larger than `sum(mask_tagged, na.rm=TRUE)` (300).

Test knit.

Logical mask indexing is a powerful way to subset data frames based on conditions, but missing values can make the results harder to interpret. When we compute `mask_tagged <- field_sightings[, "Status"] == "Tagged"`, any row where Status is NA produces NA (because `NA == "Tagged"` is unknown). That's **missing value propagation**! This is where missing values carry through comparisons as NA.

When we then subset with `tagged_only <- field_sightings[mask_tagged, ]`, R keeps rows where the mask is TRUE, but it does not treat NA as FALSE. Rows with NA in the mask can appear in the subset rows, which is why `tagged_only` has 325 rows even though there are only 300 TRUE values in the mask.

3.3.3 which() Function Helper

The function `which()` is a subsetting helper that turns a logical masks into row indices. Using `which()` is a strategy to handle potential NA values in your data. When the test could include NA, wrap it in `which()` to drop unknowns.

Use `which()` to subset the `field_sightings` data frame to keep only rows where the bird is tagged, while safely handling any potential NA values in the “Status” column.

You saw the function `which()` in Chapter 6. This function converts TRUE positions into numeric indices.

```
# Type this in your R Notebook
```

```
# Pass logic through which() to get the row indices where Status is "Tagged"
row_idx_tagged <- which(field_sightings[, "Status"] == "Tagged")
```

```
## Alternative: Create mask_tagged then pass it to which() to get row indices
# mask_tagged <- field_sightings[, "Status"] == "Tagged"
# row_idx_tagged <- which(mask_tagged)
```

Check `head(row_idx_tagged)` to confirm that `which()` returned row numbers, not TRUE/FALSE values.

```
# Check
head(row_idx_tagged)
```

```
## [1] 18 22 31 41 50 66
```

```
# Use the row indices to subset the data frame by rows that are tagged
tagged_only <- field_sightings[row_idx_tagged, ]
```

Compare `nrow(tagged_only)` to `length(row_idx_tagged)`. These should match because `row_idx_tagged` contains the exact row positions used to create `tagged_only`.

```
# Check
nrow(tagged_only)
```

```
## [1] 300
```

```
length(row_idx_tagged)
```

```
## [1] 300
```

`which()` returns the indices of the TRUE elements and omits positions where the test result is FALSE or NA values for the logical vector passed in. If the logical test can produce NA, `which()` drops unknowns.

Use a plain logical mask when the condition is simple and you have already checked how missing values behave. Use `which()` when you want row positions and want to avoid carrying NA values into the subset.

Test knit.

In conclusion, subsetting (`[ ]`) returns a piece of the original object, often preserving structure. When you need a smaller data frame, *subset*. If you need values within another function, you'll need to *extract*. In the next section, you will learn how extraction returns one component's contents.

4 Extraction

Extraction means pulling out a single component. When you want the column values themselves as a vector, operators `[[ ]]` and `$` are the clearest extraction approach. Extraction pulls out *a single component* (one column or one list element) so you can use it inside a function or calculation. Data analysts use extracted objects in a specific part of more complex code that creates a plot or summary.

4.1 `[[` Operator

There is another operator called the **double square bracket operator** (`[[ ]]`). `[ ]` is the general subsetting operator. `[[ ]]` extracts exactly one component. `[ ]` returns a subset of the same type as the original object. `[[ ]]` returns a single element inside the object (the “thing itself”), not a same-type subset. `[[ ]]` is extraction because it *pulls out* one component as the vector.

Operator	Meaning
[]	Subsetting operator. Returns a subset of the same type as the original object. Can return multiple elements.
[[]]	Extraction operator. Returns a <i>single element</i> inside the object and <i>passed</i> into functions.

Use the [[operator to extract the “Band_Color” column from the `field_sightings` data frame and then use it inside the function `unique()`

The SYNTAX: `x[[i]]`

Step 1

Extract a column as a vector

```
# Type this in your R Notebook
field_sightings[["Band_Color"]] # the column VECTOR
```

Notice: This output is long. You can use # to “comment out” the code for code chunks that might make an HTML report difficult to read.

Check that the output is a vector of values, not a 1-column data frame. You should see the band color values printed directly.

The result is an extraction from one column and returns a vector, values only.

Step 2

Use the extracted vector inside the function `unique()`

```
# Type this in your R Notebook
unique(field_sightings[["Band_Color"]])
```

```
## [1] "None" "Orange" "Silver" "Green" "Black" NA "Blue" "White"
## [9] "Red" "Yellow"
```

Check that `unique()` returns each band color only once. This confirms that `[[ ]]` gave `unique()` the column values it needed.

This shows the unique band colors in the `field_sightings` data frame. The `[[` operator allows you to extract the column as a vector, which is what `unique()` needs to work properly.

If a function needs the values inside one column, extraction is usually the better move. Ask yourself: “Does the next function need a data frame, or does it need the column values themselves?”

Test knit.

4.2 \$ Operator

There is another operator called the **dollar sign operator** (\$) that is used to extract columns from a data frame by name. It is a convenient shorthand for selecting a single column by name when you type the name literally.

The SYNTAX:

```
dataframe$column_name
```

Extract the `Band_Color` column using `$`.

```
# Type this in your R Notebook
field_sightings$Band_Color
```

Check that `$` returns the same kind of vector output as `field_sightings[["Band_Color"]]`.

A vector is printed showing all element values from the column `Band_Color` from the `field_sightings` data frame. It is an extraction from one column and returns a vector, values only.

You will likely use the `$` operator often. The `$` operator is more concise and easier to read for common uses cases. However, it only works for selecting one column at a time and can't use an assigned variable stored in the environment.

If we try to use a variable to specify the column name with the `$` operator, it will not work because `$` does not evaluate the variable name. `[[ ]]` can use a variable and is safer in programs.

Use a stored *object* name to specify the column name and try to extract the column using both `[[` and `$` operators.

```
# Type this in your R Notebook
# Example 1
col_name <- "Band_Color"
field_sightings[[col_name]]    # works
```

The `[[` operator allows you to use a variable name from your environment to specify the column name, which is why it works in this case. The result is an extraction from one column and returns a vector, values only.

```
# Type this in your R Notebook
# Example 2
col_name <- "Band_Color"
field_sightings$col_name      # looks for a column literally named "col_name"
```

```
## Warning: Unknown or uninitialised column: `col_name`.
```

```
## NULL
```

Check the output. It should return `NULL` because there is no column named `col_name` in `field_sightings`.

The `$` operator looks for a column literally named “`col_name`” and finds none, resulting in `NULL`.

This is a method-choice decision. Use `$` when you are typing the column name directly. Use `[[ ]]` when the column name is stored in an object. If your code uses a stored column name and returns `NULL`, check whether you used `$` when you needed `[[ ]]`.

Test knit.

So far, you've learned two different moves:

- Subset: `field_sightings["Band_Color"]` (data frame)
- Extract: `field_sightings[["Band_Color"]]` or `field_sightings$Band_Color` (values)

Next, we'll learn how to change values inside an existing object using replacement methods.

5 Subassignment (replacement)

Subassignment means replacing part of an object (vector, matrix, data frame, list, etc.) with new values using indexing. It is an umbrella term for *assigning* into part of an object using an index on the left of `<-`.

In this chapter, you'll use three core replacement patterns for data frames:

1. Use `x[i, j] <- value` to change specific rows and columns.

2. Use `x[[...]] <- value` to replace/add one column (works when the column name is stored in a variable).
3. Use `x$name <- value` to replace/add one column (when the column name is typed literally).

Indexing on the left of `<-` edits the object you assign into

5.1 Overwrite with `x[i, j] <- value`

Overwrite in an informal type of replacement where you change existing values.

The SYNTAX:

```
x[i, j] <- value
```

Notice that indexing is located on the left side of an assignment operator (`<-`) to change specific values in a data frame.

What if all the tagged birds in the `field_sightings` data frame were raised as part of a breeding program and then reintroduced into an area where the species no longer existed? This means all the untagged birds were hatched in the wild. Suppose we decide to rename the element value "Untagged" to "Wild Hatch" in the `Status` column to communicate this information more clearly.

Replace "Untagged" with "Wild Hatch" in `Status`.

Step 1

Make a copy of the data frame.

```
# Type this in your R Notebook
reintroduction <- field_sightings           # copy
```

Check the Environment pane. You should now see both `field_sightings` and `reintroduction`.

Making a copy is a protective habit. It gives you the freedom to experiment because the original data frame is still available for comparison if your replacement code does something unexpected.

We will overwrite values using the copy dataframe, `reintroduction`.

Step 2

Get row indices with `which()` to identify the rows where `Status` is "Untagged". `which()` returns the row positions where the test is `TRUE` and ignores positions where the test is `FALSE` or `NA`. Use the dataframe that you want to overwrite values in, not the original data frame.

```
# Type this in your R Notebook
indices_untagged <- which(reintroduction[, "Status"] == "Untagged")
```

Run `head(indices_untagged)` to confirm that `which()` returned row numbers. These are the row positions where `Status` was "Untagged".

```
# Check
head(indices_untagged)
```

```
## [1] 1 2 3 4 5 6
```

Step 3

Overwrite using `df[rows, "col"] <- value`

```
# Type this in your R Notebook
reintroduction[indices_untagged, "Status"] <- "Wild Hatch"
```

Step 4

Verify that the values have been changed.

Compare the two outputs. `reintroduction` should include “Wild Hatch”. The original `field_sightings` should still include “Untagged”.

```
# Check
unique(reintroduction[, "Status"])      # modified data frame
```

```
## # A tibble: 3 x 1
##   Status
##   <chr>
## 1 Wild Hatch
## 2 Tagged
## 3 <NA>
```

```
unique(field_sightings[, "Status"])    # original data frame
```

```
## # A tibble: 3 x 1
##   Status
##   <chr>
## 1 Untagged
## 2 Tagged
## 3 <NA>
```

The `Status` column in the `reintroduction` data frame has been modified to replace “Untagged” with “Wild Hatch”.

Test knit.

5.2 Insertion

Insertion is an informal type of replacement where you add something new (most commonly a new column).

`x[...]` and `x$name <- value` replace (or create) one component. For data frames, that’s typically a single column.

Suppose we decide to add a new column called `Tag_Flag` that indicates whether the bird is tagged or not. It is common to use binary columns to create figures or run analyses. We want to set the value to 1 for tagged birds and 0 for untagged birds so we can later create a visualization of how many tagged vs. untagged birds we have.

5.2.1 [[Method

Add a new column (a flag) using a column name stored in a variable.

Step 1

Make a copy of the data frame.

```
# Type this in your R Notebook
reintroduction2 <- field_sightings
```

Check that `reintroduction2` appears in the Environment pane. This is the copy you will modify.

Step 2

Store the new name of your column a variable

```
# Type this in your R Notebook
new_col <- "Tag_Flag"
```

Check that `new_col` contains the text "Tag_Flag". This object stores the new column name.

Step 3

Add the new column using `[[ ]]` <-

```
# Type this in your R Notebook
# Start everyone as 0
reintroduction2[[new_col]] <- 0          # <- Uses [[]]

# Change to 1 where Status is Tagged
idx_tagged <- which(reintroduction2[, "Status"] == "Tagged")
reintroduction2[idx_tagged, new_col] <- 1
```

Step 4

Verify that the new column has been added and values are correct.

Run `names(reintroduction2)` and confirm that `Tag_Flag` appears as a new column name.

```
# Check
names(reintroduction2)

## [1] "Status"      "Band_Color" "Tag_Flag"
```

Feel free to click `reintroduction` and `reintroduction2` in your Environment pane to view the data frame and compare

5.2.2 \$ Method

If you are typing the column name directly (not stored in a variable), you can use `$` to add the column.

Add a new column (a flag) using a column name typed literally with `$`.

Step 1

Make a copy of the data frame.

```
# Type this in your R Notebook
reintroduction3 <- field_sightings
```

Check that `reintroduction3` appears in the Environment pane. This is the copy you will modify.

Step 2

Add the new column using `$` <-

```
# Type this in your R Notebook
# Start everyone as 0
reintroduction3$Tag_Flag <- 0          # <- Uses $

# Change to 1 where Status is Tagged
idx_tagged <- which(reintroduction3[, "Status"] == "Tagged")
reintroduction3[idx_tagged, "Tag_Flag"] <- 1
```

Step 3

Verify that the new column has been added.

Run `names(reintroduction3)` to confirm that `Tag_Flag` was added.

```
# Check
names(reintroduction3)
```

```
## [1] "Status"      "Band_Color" "Tag_Flag"
```

Use `[[ ]]` when the column name is stored in a variable (like `new_col`). Use `$` when you are typing the column name literally.

Choose `$` when the new column name is typed directly into the code. Choose `[[ ]]` when the column name is stored in an object. Both methods can work, but they are not interchangeable in every situation.

6 Code Chunk Options

You probably noticed that some code chunk examples in this chapter resulted in long vector outputs. These can become difficult to read and navigate. You might find them particularly annoying when reviewing your HTML report. This final section explains how to manage long outputs in your R Notebook.

To make your R Notebook more readable, you can use the `results = 'hide'` option in your code chunks to hide the output of certain chunks, especially those that produce long outputs.

Alternatively, you can use `eval = FALSE` to prevent the code from running at all, which is useful for code that is meant to be illustrative rather than executed, such as when there is an error that you want to discuss without actually running the code.

Below are some examples of how to use these options in your R Notebook. Try out the code chunk settings in your notebook to see how they work!

```
```{r, echo=TRUE, results='hide'}
Show code, hide long output
field_sightings[['Band_Color']]
```

```{r, echo=FALSE, message=FALSE, warning=FALSE}
Run, but don't show code/messages/warnings
field_sightings$col_name
```

```{r, eval=FALSE, echo=TRUE}
Show code, but do not run it
field_sightings$Band_Color
```
```

Experiment with these options in your R Notebook to find the best way to manage long outputs above to make your notes and practice reports more readable. Knit your notebook to HTML to see how it looks, then adjust the options and other formatting and re-knit to make sure your report is professional and ready to share with your research collaborators.

Knit your notebook and look at the HTML output. Confirm whether the code, output, messages, and warnings appear or are hidden the way you intended.

Code chunk options are report-design decisions. Use them to make your notebook easier to read, not to hide evidence that your code failed. If a warning or message helps explain what happened, keep it visible or memo it.

7 Summary

In Chapter 8, you learned how to use base R indexing to select, extract, and modify parts of a data frame. These skills are useful because most analysis does not use every row and every column at the same time. You need to be able to point R at the exact part of the data you want and check that R returned what you intended.

You practiced subsetting with `[ ]` using the `df[rows, cols]` pattern. You selected rows and columns by position, selected columns by name, and used logical masks to keep rows that met a condition. You also learned that the output can change depending on how you index. Some code returns a data frame, while other code returns a vector, and that difference matters for what you can do next.

You also practiced extraction with `[[ ]]` and `$`. Extraction pulls out one column as a vector, which is useful when you want to pass the column values into functions such as `unique()` or `table()`. You learned that `[[ ]]` is safer when the column name is stored in an object, while `$` is convenient when you type the column name directly.

Logical masks introduced an important warning about missing values. When a comparison involves `NA`, the result can also be `NA`. That means a logical mask can contain `TRUE`, `FALSE`, and `NA`, which can make subsetting results harder to interpret. You practiced using `which()` to convert `TRUE` values into row positions and avoid carrying unknown values into the subset.

Finally, you practiced subassignment by making a copy of a data frame, finding row positions with `which()`, and overwriting selected values. You also practiced adding a new column. These are important data-cleaning skills because real datasets often need corrections or new variables before they are ready for analysis.

The main habit to carry forward is this: after indexing, extracting, or modifying data, verify what changed. Check the dimensions, names, object type, values, or summary output before moving on. In the next chapter, you will extend this work by filtering with more complex conditions using `&`, `|`, and `%in%`.

8 Chapter Terms

Column index: The index used to select columns in a data frame or matrix (the second slot inside brackets: `x[, j]`).

Conditional Subsetting (Logical Subsetting): Filtering an object by using a logical condition inside `[` (e.g., `df[df$cyl != 4, ]`), where `TRUE` keeps an element/row and `FALSE` drops it. This contrasts with negative indexing, which excludes by position rather than by a condition.

Dollar Sign Operator (`$`): An extraction (and replacement) operator used to access a single named component of a recursive object. Most commonly a data frame column (`df$col`) or a list element (`lst$name`). `$` requires a literal name (not a name stored in a variable), returns `NULL` if the name doesn't exist, and by default may do partial matching (which can surprise you).

Double Square Bracket Operator (`[[ ]]`): An extraction operator used to pull out one single element from a list (or one column from a data frame) and return the element itself, not a same-type subset. Unlike `[`, which can return multiple elements and preserves the container type, `[[ ]]` is for “give me the one thing inside” (e.g., a vector inside a dataframe).

Equal To (`==`): A relational operator that tests equality and returns `TRUE` where values are equal and `FALSE` where they are not (element-by-element for vectors).

Extraction: Retrieving a single component from an object—such as one list element or one data-frame column—so you get the “thing itself” (often a simplified result like a vector). Extraction is commonly done with `[[ ]]` (extract one element by position or name) or `$` (extract one column by name).

Indexing Operator: The square-bracket indexing operator `[` or `[[ ]]` are used to extract or replace parts of an object in R (for example, `x[i]`). For vectors, it selects one or more elements and returns the same kind of object as `x` (a vector).

Insertion: A common type of subassignment where you add something new to an object—most often a new column in a data frame. Example: `df[["Tag_Flag"]] <- 0` (creates the column if it doesn't exist yet).

Less Than or Equal To (`<=`): A relational operator that returns `TRUE` where the left side is less than or equal to the right side, and `FALSE` otherwise.

Logical Indexing: Indexing a vector using a logical vector of `TRUE`/`FALSE` values (for example, `x[x > 0]`) so R keeps the elements where the index is `TRUE` and drops those where it is `FALSE`. If the logical index is a different length, R may recycle it, which can lead to surprising results.

Missing Value Propagation: R's behavior where an `NA` (unknown value) “carries through” computations and comparisons. If an operation can't be determined because of missing data, the result is often `NA` (unknown) instead of `TRUE` or `FALSE`. For example, `NA == "Tagged"` returns `NA`. If you use a logical mask that contains `NA` values for subsetting (e.g., `x[mask]` or `df[mask, ]`), R does not treat `NA` as `FALSE`, so the subset may include `NA` values (or `NA` rows), which can change counts and results.

Names Indexing: Indexing a named vector using character strings that match element names (for example, `x[c("Octarine", "TARDIS Blue")]`). With `[`, names are matched exactly; any name that doesn't exist returns `NA` for that position.

Overwrite: A common type of subassignment where you change existing values (you replace what is already there). Example: `df[rows, "Status"] <- "Wild Hatch"`.

Positional Indexing: Indexing a vector using numeric positions (for example, `x[1]` for the first element or `x[c(1, 3)]` for the first and third). Positions are based on the current order of the vector.

Row index: The index used to select rows in a data frame or matrix (the first slot inside brackets: `x[i, ]`).

Subassignment: Replacing part of an object by putting an index on the left side of `<-`. R changes only the indexed part of the object. Examples: `x[i] <- value`, `df[rows, "col"] <- value`, `df[[col]] <- value`, `df$col <- value`.

Subsetting: Selecting a smaller piece of an R object using indices (positions, names, or a logical mask). With data frames, subsetting most commonly uses `[` in the form `df[rows, cols]` and typically preserves the object's structure (e.g., you still get a data frame when you subset rows and columns).

9 Practice Space

This Practice Space uses a new dataset to help you generalize your learning. While the column names differ, the indexing patterns are the same. You will work with `Values_Transport_dplace.csv` focusing on the columns `id`, `name`, `sub_case`, and `year`.

Open a new R Notebook and save it in your course folder as: `chapter8_practice.Rmd`.

Fill in the blanks and answer the short prompts in your R Notebook. This Practice Space helps you check whether you can organize a notebook, read an R help page, use a function argument, and knit your work into an HTML report.

Before you begin:

1. Create a new R Notebook in RStudio.
2. Save it in your course folder as `chapter8_practice.Rmd`.
3. Edit the YAML so that it includes:
 - the title “Chapter 8 Practice”,
 - your name as the author, and
 - the date.
4. Remove the default content in the R Notebook.

5. Create heading for each task (1:9) using the **##** heading level.

Complete each task in order.

Add short memo notes throughout your notebook. Your memo notes should explain what your code does, what output you expected, what surprised or confused you, and what you would check next if something did not work.

In this Practice Space, you will see two kinds of memo prompts:

- **Add Comments** asks you to explain what the code or output shows.
- **Reflect** asks you to explain your thinking process, what confused you, what changed in your understanding, or what strategy you would use next.

These are there to help you practice the habit of writing memos as you work. **In each task, you can choose whether to answer one, two, or all memo prompts.** Each task should contain at least one memo response. The goal is to practice writing your thinking as you work, not to write a perfect answer for every prompt.

Autonomy is the habit of taking ownership of your thinking, choices, code, and learning process while using help responsibly. Show autonomy by checking your output, deciding what evidence shows your code worked, and explaining what you would check next if something did not work. If you encounter an error, document your **tenacity** by briefly noting what happened, what you tried, and how you fixed it or what still confuses you. Document how you troubleshooted the error and what resource(s) you used to help you resolve the issue (**transparency**). Show **courage** by sharing your thinking process, even if it includes mistakes or confusion.

You may quote AI output only when you clearly mark it as quoted text, either with quotation marks or a block quote using `>`. Then hand-type your own explanation of why you are including the quote.

Before you finish, clear your Environment and run the notebook from top to bottom. Then knit to HTML to check that the notebook works and is formatted correctly.

9.1 Task 1

Load the CSV with `readr::read_csv()`

Load `Values_Transport_dplace.csv` into R using `read_csv()` and assign it to a variable called `values_transport`.

Check the output from `list.files()` and confirm that `Values_Transport_dplace.csv` appears exactly as written.

```
# Type this in your R Notebook, Fill in the blanks
# 1) Confirm the CSV is in the same folder as your R Notebook
list.files()

# Type this in your R Notebook, Fill in the blanks
# 2) Load required package, then read the CSV
library(____)

_____ <- read_csv(__Values_Transport_dplace.csv__)
```

Check that `values_transport` appears in your Environment pane and that it is a data frame.

Add Comments: Explain what evidence showed you that the CSV file loaded successfully.

Reflect: If `values_transport` failed to be added to your environment, what would you check first?

Test knit.

9.2 Task 2

Positional subsetting (rows/cols by index number)

Create `first_10_basic` containing the first 10 rows and columns 2 through 5 of `values_transport`. Use `df[rows, cols]`

```
# Type this in your R Notebook, Fill in the blanks
first_10_basic <- _____[____, ____]

# Check
dim(first_10_basic)
head(first_10_basic)
```

Check that `first_10_basic` has 10 rows and columns 2 through 5.

Add Comments: Explain what the row index selected and what the column index selected in your code.

Reflect: Why is checking the dimensions useful after subsetting? What could go wrong if you selected the wrong rows or columns and did not check?

Test knit.

9.3 Task 3

Names/character subsetting (choose specific columns by name)

Create `id_name_year` containing only the columns: `id`, `name`, and `year`.

```
# Type this in your R Notebook, Fill in the blanks
_____ <- values_transport[, c(____, ____, ____)]

# Check
names(id_name_year)
head(id_name_year)
```

Add Comments: Explain how the output from `names(id_name_year)` confirms that your names indexing worked.

Reflect: Why might names indexing be safer than positional indexing when selecting columns from a data frame?

Test knit.

9.4 Task 4

Logical mask subsetting

Subset rows where `year` greater than or equal to 1900 using `which()` to ensure not NA problems.

```
# Type this in your R Notebook, Fill in the blanks
idx_1900 <- _____(values_transport[, _____] __ 1900)

values_1900 <- values_transport[_____, ]
```

Check that `idx_1900` contains row positions and that `length(idx_1900)` matches `nrow(values_1900)`.

```
# Check
head(idx_1900)
length(idx_1900)
nrow(values_1900)
```

Add Comments: Report how many entries have a year of 1900 or later. Explain which output gave you that number.

Reflect: Why does this task use `which()` instead of putting the logical condition directly inside the row position? What problem is `which()` helping you avoid?

Test knit.

9.5 Task 5

Explain Missing Value Propagation

In 3–5 sentences, explain what **Missing Value Propagation** means when you create a logical mask like:

```
mask <- values_transport[, "year"] >= 1900
```

Check that `mask` is a logical vector. It should contain `TRUE`, `FALSE`, and possibly `NA`.

```
# Check  
unique(mask)
```

Reflect: In 3–5 sentences, explain what missing value propagation means when you create a logical mask like `mask <- values_transport[, "year"] >= 1900`. What will R return when a comparison involves `NA`, such as `NA >= 1900`? Why can `NA` values in a logical mask cause confusing subsetting results with `[mask, ]`? How does `which()` help avoid this issue?

Test knit.

9.6 Task 6

Extraction

Extract the `year` column as a vector using `[[ ]]`, storing it as `year_vector`.

```
# Type this in your R Notebook, Fill in the blanks  
_____ <- values_transport__"year"__
```

Check that `year_vector` is a vector, not a one-column data frame.

```
# Check  
table(_____  
unique(_____)
```

Add Comments: Comment on the output of `table(year_vector)` or `unique(year_vector)`. What do you notice about possible values in the year column?

Reflect: Why does extraction with `[[ ]]` make sense here? What kind of object does `table()` need: a data frame or the values inside one column?

Test knit.

9.7 Task 7

Overwrite

Scenario: You found four data-entry errors in the year column. After reviewing field notes, you confirm these corrections:

- -621 should be 1821
- -1750 should be 1750
- 110 should be 1910

- 1292 should be 1892

Use subassignment with `df[rows, "col"] <- value` to overwrite the incorrect year values in the copy of `values_transport` data frame called `transport_year_clean`.

```
# Type this in your R Notebook, Fill in the blanks
# Step 1: Make a copy of the data frame
_____ <- _____

# Step 2: Identify row indices for each incorrect value
idx_621 <- _____(transport_year_clean[, "year"] == -621)
idx_1750 <- _____(transport_year_clean[, "year"] == -1750)
idx_110 <- _____(transport_year_clean[, "year"] == 110)
idx_1292 <- _____(transport_year_clean[, "year"] == 1292)

# Check
idx_621
idx_1750
idx_110
idx_1292

# Step 3: Overwrite the incorrect values with the correct ones
transport_year_clean[_____, "year"] <- 1821
transport_year_clean[_____, "year"] <- 1750
transport_year_clean[_____, "year"] <- 1910
transport_year_clean[_____, "year"] <- 1892
```

Compare the unique year values in `transport_year_clean` to the original `values_transport` to confirm that the incorrect values have been replaced and that the original data frame still contains the incorrect values.

```
# Check
unique(transport_year_clean[["year"]])
unique(values_transport[["year"]])
```

Add Comments: Explain why you made a copy before overwriting values and why `which()` was useful for finding the row positions.

Reflect: What would you check if one of the incorrect year values still appeared after your replacement code? What would you check if the original `values_transport` changed too?

Test knit.

9.8 Task 8

Save your workspace (`chapter8_practice.RData`).

Your environment should contain:

- `values_transport` (original data frame loaded from CSV)
- `first_10_basic` (subset with first 10 rows and columns 2-5)
- `id_name_year` (subset with only `id`, `name`, `year` columns)
- `values_1900` (subset with rows where `year` $\geq$ 1900)
- `transport_year_clean` (data frame with corrected year values)
- `mask` (logical vector for `year` $\geq$ 1900)

If you have extra objects in your environment, you can click the **broom icon** to clear them. Then re-run all chunks and verify environment, and then save your workspace.

Option 1. Use `save.image()` to save your workspace as `chapter8_practice.RData` in your course folder.

```
# Type this in your R Notebook
save.image("chapter8_practice.RData")
```

Option 2. Press the **disk icon** in the Environment pane to save your workspace.

Check that `chapter8_practice.RData` appears in your module folder.

Reflect: Why is it useful to save both the `.Rmd` notebook and the `.RData` workspace? What does each file show about your work?

9.9 Task 9

Create a well-formatted HTML report for submission.

Step 1: Sweep your Environment (start clean).

Click the **broom icon** in the Environment pane to clear objects.

Step 2: Run everything top-to-bottom.

Use **Run** → **Run All** to execute every chunk in order.

Check that the notebook runs from top to bottom without errors after you clear the Environment. This confirms that your notebook contains the code needed to rebuild your work.

Step 3: Save

After your notebook runs without errors, save your notebook (`.Rmd`) and knit to HTML (`.HTML`).

After you knit, look in your course folder for two files with the same name. One file ends in `.Rmd` and one ends in `.html`.

The HTML usually has a internet browser icon. You can check which is which by double clicking to open them. One opens in R and the other usually opens in a web browser. Upload the `.html` file. Before submission, check that the uploaded file ends in `.html`.